Big-O

Arrays
Dynamic Arrays
Linked lists
- SLL
- DLL
- Circular Linked lists

Stacks
Queues
Binary Search Trees
Balanced BSTs
- AVL Trees
- Redblack trees

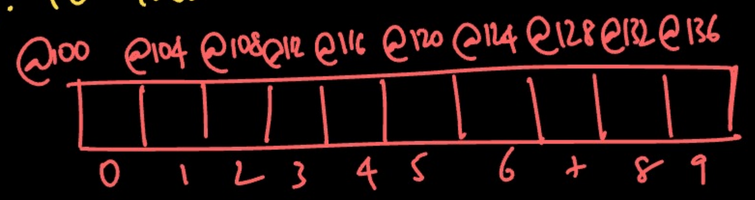
Heaps
Hash Tables (maps & sets)
Graphs

Tries
Union Find Disjoint Sets

Algos
Searching
Sorting
Graph algos
String

Arrays

- int - 4 bytes
- All the elements in the array stored as one contiguous block
- Eg: 10 ints



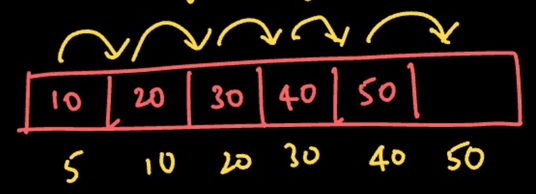
Randomly accessing an element : arr[i]

$$\underbrace{\text{Base addr}}_{@100} + \underbrace{\text{index}}_{5} \times \underbrace{\text{int size}}_{4} = @120$$

Very fast
O(1) time complexity
Biggest advantages.

Disadvantages

- Size is fixed at creation
- Insert a new element at the beginning/middle



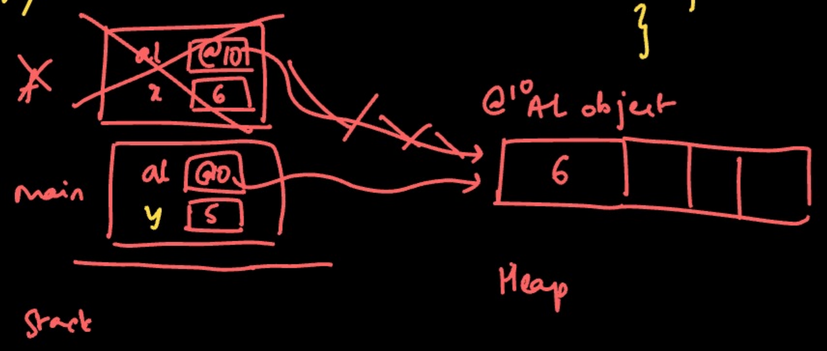
Running time : $O(n)$

$$3n^2 + 5n + 17 \rightarrow O(n^2)$$

Insert at end of array : $O(1)$
(nothing to move)

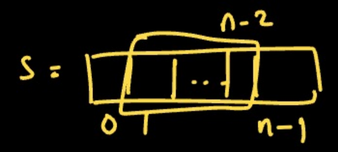
```
void f ( AL<Integer> al, int x ) {
    x = x + 1;
    al.add(x);
}

main() {
    AL<Integer> al = new AL<>();
    int y = 5;
    f(al, y);
    print(al);  // {6}
    print(y);   // 5
}
```



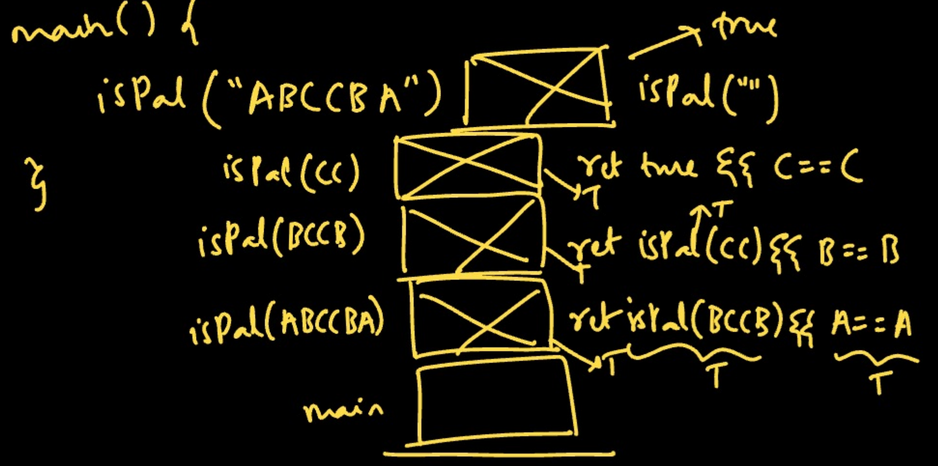
Eg:

```
boolean isPalindrome (String s) {
    if (s.length() == 1 or 0) {
        return true;
    }
    else {
        String s-sub = ...;
        return isPalindrome(s-sub) && s[0] == s[n-1];
    }
}
```



M ADAM

s-sub = s [1 : n-2]

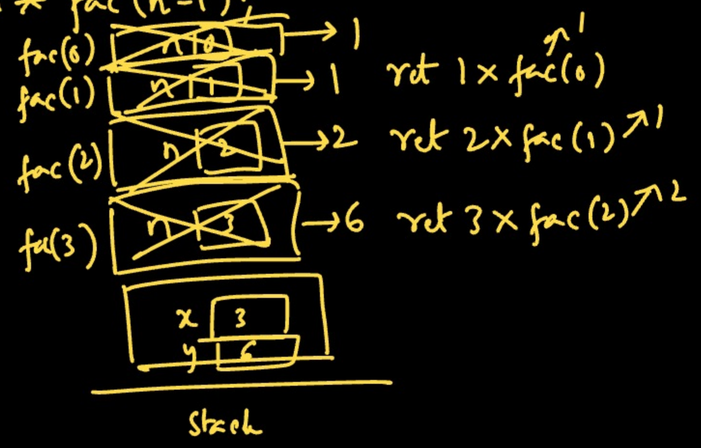


```
int fac (int n) {
    int prod = 1;
    for (i=1 ; i<=n ; i++) {
        prod = prod * i;
    }
    return prod;
}
```

$n! = n \times (n-1)!$

```
int fac (int n) {
    if (n==0) return 1;
    else return n * fac(n-1);
}

main() {
    int x = 3;
    y = fac(3);
}
```



Big O

```
int fac(int n) {
    int prod = 1;              // 1 step
    for (i=1; i<=n; i++) {     // n times
        prod = prod * i;
    }
    return prod;               // 1 step
}
```

1. i<=n check - 1 step
2. prod * i - 1 step
3. prod = prod * i - 1 step
4. i++ - 1 step

$1 + 4n + 1 = 4n + 2$ steps

- Asymptotic Analysis
  - We consider only very high values of n ⇒ As $n \to \infty$

Algo1 : $100n$

Algo2 : n^2 (quadratic/ polynomial) 2^n - exponential

$4n^2 + 3n + 7 = O(n^2)$

$3n\log n + 7n^2 = O(n^2)$

$\log n \ll n$

$n\log n \ll n^2$

(4 times)

$16 \to 8 \to 4 \to 2 \to 1$

$\log_2 16$ → how many times we have to divide 16 by 2 to get to 1.

$e = 2.71\ldots$

$\log_{10} n = \dfrac{\log_e n}{\boxed{\log_e 10}}$ constant

$\log_{10} n = O(\log_{10} n)$

$\log_{10} n = \dfrac{\log_2 n}{\log_2 10} = O(\log_2 n)$

$f(n) = 10n^2 + 7$

$f(n) = O(g(n))$

if there are numbers N & c such that

for all $n \geq N$ $\quad f(n) \leq c \cdot g(n)$

$10n^2 + 7 = O(n^2)$

$$\boxed{10n^2 + 7 \leq c \cdot n^2 \text{ for all } n \geq N}$$

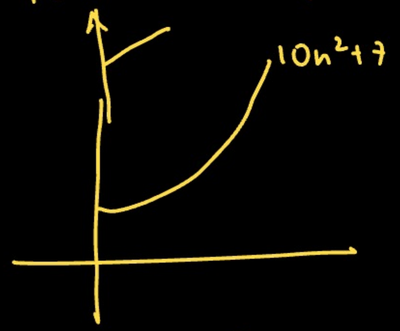
for all $n \geq 1$ $\quad 10n^2 \leq 10n^2$

for all $n \geq 1$ $\quad 7 \leq 7n^2$

for all $n \geq 1$ $\quad 10n^2 + 7 \leq 17n^2$

$N = 1$
$c = 17$

$10n^2 + 7 \rightarrow O(n)$



$10n^2 + 7 \leq 1000 \cdot n$

for $n \geq 1$

```
boolean hasDuplicate(int[] arr) {
    for (i=0; i<n; i++) {
        for (j=i+1; j<n; j++) {
            if (arr[i]==arr[j])
                return true;
        }
    }
    return false;
}
```

$\boxed{n = 5}$

$i = 0$

j goes from 1 to $n-1$ (n-1 times) 1 step } n-1 steps

$i = 1$

j goes from 2 to $n-1$ 1 step } n-2 steps

⋮

$i = n-1$

j goes $(n-1)+1$ n to $n-1$ (0 times) } 0 steps

$(n-1) + (n-2) + \cdots + 1 + 0$

$= \dfrac{n(n-1)}{2} = \dfrac{1}{2}n^2 - \dfrac{1}{2}n = O(n^2)$

$S = 1 + 2 + 3 + \cdots + n$

$S = n + (n-1) + (n-2) + \cdots + 1$

$2S = (n+1) + (n+1) + (n+1) + \cdots + (n+1)$

Sum of first k natural nums $= \dfrac{k(k+1)}{2}$

" " " $k-1$ " " $= \dfrac{(k-1)(k-1+1)}{2}$

Recursion

- Breaking down a problem into a smaller problem of the same type
- Base case
- Eg: $n! = n \times (n-1)!$

$5! = 5 \times \boxed{(4!)}$

$sum(arr[0...n-1]) = arr[0] + \boxed{sum(arr[1...n-1])}$

$reverse("ABCD") = \underbrace{reverse("BCD")}_{"DCB"} + "A"$

B Case:

$reverse("A") = "A"$

$fac(5) = 5 \times \underbrace{fac(4)}_{24}$

$numVowels("ABCDE") = \underbrace{numVowels("BCDE")}_{1} + (isVowel(A)\ ?\ 1 : 0)$

B Case

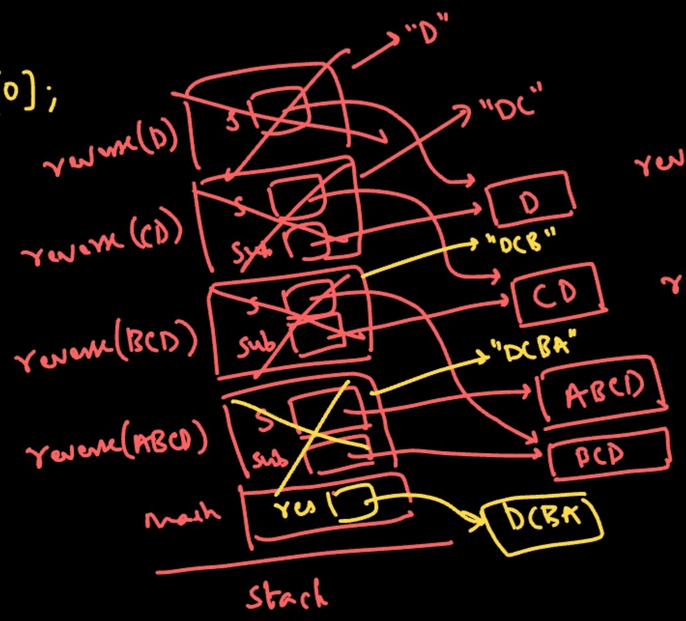
```
if (len==1)
    return ifVowel(s[0]) ? 1 : 0;
```

```
int sum(int[] arr) {
    if (arr.length.l==1)
        return arr[0];
    else {
        int[] subarray = arr[1...n-1];
        return arr[0] + sum(subarray);
    }
}
```

```
String reverse (String s) {
    if (s.length() ==1)
        return s;
    else {
        String substring = s[1...n-1];
        return reverse(substring) + s[0];
    }
}
```

```
String reverse (String s) {
    if (s.length() ==1)
        return s;
    else {
        String substring = s [1...n-1];
        return reverse (substring) + s[0];
    }
}

main() {
    String res = reverse("ABCD");
    print(res);
}
```



reverse(D) + s[0] = DC
D → C

reverse(CD) + s[0] = DCB
DC → B

reverse(BCD) + s[0] = DCBA
DCB → A

reverse (ABCD) = D + $\underbrace{\text{reverse (ABC)}}_{\text{CBA}}$

isPalindrome(s) = (s[0] == s[n-1]) AND isPalindrome (s[1..n-2])

↓

Base:
len==1 or len==0
return true

$\underbrace{\text{M ADAM}}$

ABBA
↑ ↑
BB
↑ ↑
""

Mathematical Induction

- Prove Sum of first 'n' natural numbers: $\frac{n(n+1)}{2}$

$$S = 1 + 2 + 3 + \cdots + n$$
$$S = n + (n-1) + (n-2) + \cdots + 1$$

$$2S = (n+1) + (n+1) + (n+1) + \cdots + (n+1)$$
$$2S = n(n+1)$$
$$\boxed{S = \frac{n(n+1)}{2}}$$

Proved:

1. $S(1)$ works
2. If $S(k)$ works, then $S(k+1)$ works

$S(1)$ - works (BC)

If $S(1)$, then $S(2) \Rightarrow S(2)$ works

If $S(2)$, then $S(3) \Rightarrow S(3)$ works

Base case ✓

reverse("A") → A

$$\underbrace{\text{reverse}(ABCD)}_{k+1} = D + \underbrace{\text{reverse}(ABC)}_{\underbrace{CBA}_{k}}$$

To prove $\boxed{S(n) = \frac{n(n+1)}{2}}$

Steps

1. prove it for the base case

$$S(n) = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

$n = 1$ for base case

$$S(1) = 1$$
$$\frac{n(n+1)}{2} = \frac{1 \times 2}{2} = 1$$

2. Assume that it is true for $n = k$

& then prove that is true for $n = k+1$

Assume $S(k) = \frac{k(k+1)}{2}$ it is true

To prove $\boxed{S(k+1) = \frac{(k+1)(k+2)}{2}}$

$$\text{LHS} = S(k+1) = 1 + 2 + 3 + \cdots + (k+1)$$
$$= S(k) + (k+1)$$
$$= \frac{k(k+1)}{2} + (k+1)$$
$$= \frac{k(k+1) + 2(k+1)}{2}$$
$$= \frac{(k+1)(k+2)}{2} = \text{RHS}$$

Big O

- Asymptotic analysis
  (we only consider large values)

$f(n) = O(g(n))$

only if there exists constants c, N such that

$f(n) \le c \cdot g(n)$ for all $n \ge N$

$\underbrace{n^3+3n}_{f(n)} = O(\underbrace{n^2}_{g(n)})$?

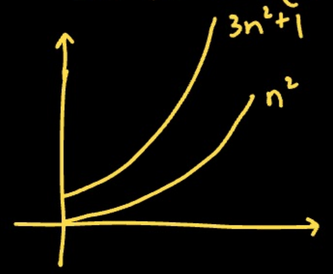
Eg: $f(n) = 100n$
$g(n) = n^2$

$100n = O(n^2)$

$100n \le 1 \cdot n^2$ for all $n \ge 100$

$\boxed{c=1, N=100} \Rightarrow \boxed{100n = O(n^2)}$

Eg: $3n^2+1 = O(n^2)$



$3n^2 \le 3n^2$ for all $n \ge 1$

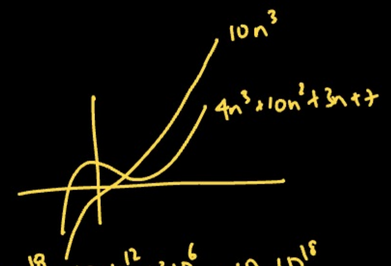
$1 \le 1n^2$ for all $n \ge 1$

$\boxed{3n^2+1 \le 4n^2 \text{ for all } n \ge 1}$

$\Rightarrow \boxed{c=4, N=1}$ $3n^2+1 = O(n^2)$

$3n^2+1 \le 1000n^2$ for $n \ge 1$

$c = 1000$
$N = 1$



$4 \cdot 10^{18} + \underbrace{10 \cdot 10^{12} + 3 \cdot 10^{6} + 7}_{} \le 10 \cdot 10^{18}$

$4n^3+10n^2+3n+7 \le 10n^3$ $\boxed{c=10, N = \text{1million}}$

$\le 4n^3+10n^3+3n^3+7n^3$ $N+1$ $N+2$

$\le 24n^3$

$4n^3+10n^2+3n+7 \le 24n^3$ for $n \ge 1$

$\boxed{c=24, N=1}$

$4n^3+10n^2+3n+7 = O(n^3)$

$4n^3+10n^2+3n+7 \le cn^3$

(1) $100n \le 2 \cdot n^2$ for all $n \ge 200$

$100n = O(n^2)$

$\boxed{\begin{array}{l} C = 2 \\ N = 200 \end{array}}$

(2) $4n^3 + 10n^2 + 3n + 7 \le 24n^3$

for all $n \ge 1$

$\boxed{4n^3 + 10n^2 + 3n + 7 = O(n^3)}$

$\boxed{\begin{array}{l} C = 24 \\ N = 1 \end{array}}$

(3) $100n^3 = O(n^3)$

$100n^3 \le 101n^3$ for all $n \ge 1$

$\boxed{\begin{array}{l} C = 101 \\ N = 1 \end{array}}$

$100n^3 = O(n^3)$

(4) $n^2 = O(\underbrace{1000000\,n}_{g(n)})$

Is there $c; N$

such that after a $n \ge N$

$\boxed{n^2 \le c \cdot 1000000\,n}$ No !!

$n^2 \ne O(n)$ or $O(1000000n)$

1. $f(n) = O(g(n))$
2. equivalent to saying "$g(n)$ is the upper bound of $f(n)$"
3. equivalent to saying for some constants c, N

   $f(n) \le c \cdot g(n)$ for all $n \ge N$
4. equivalent to saying

   $c \cdot g(n)$ is <u>always</u> greater than or equal to $f(n)$ after a certain point.

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$

$< O(n^3) \cdots < O(2^n)$

2^n grows faster than even n^{1000}

1.1^n grows faster than even $n^{1000000}$

$1.1^n \ne O(n^{1000000})$

```
void f (String s) {
    if (s.len == 0) return;
    print(s);
    String sub = s[0...n-2]; //ABC
    f(sub);
    print (s);
}
```

s - ABCD

ABCD
ABC
AB
A
A
AB
ABC
ABCD

ABCD
ABC
AB
A
A
AB
ABC
f(ABC
ABCD

f(ABCD)
f(ABC)
f(AB)
f(A)
f("")
f(A) → A
f("")
f(A) → A
f("")
f(A) → A

ABCD
ABC
AB
A

```
total = 0
for (int i=0; i<n ; i++) {
    total += i;
}
```

// 1 step
// i=0 1 step

n times { i<n 1 step
t+i 1 step
t=t+i 1 step
i++ 1 step

i becomes n i<n - 1 step

$2 + 4n + 1 = 3+4n$ → function of n

assignment	- 1 step
addn/sub	- 1 step
mul/div	- 1 step
comp	- 1 step

$f(x) = x^2 + 2$

$x^2 + xy$
fn of x & y

$f(n) = 3+4n$ $g(n)$
→ $O(g(n))$

$3+4n = O(n)$ ✓

$3+4n = O(n^2)$ ✓ $c \cdot n \geq 3+4n$ always after a certain pt

$3+4n = O(\log n)$ ✗

Big O - tells us about the upper bound

$\log_2 16 = 4$

16 → 8 → 4 → 2 → 1

$\log_3 27 = 3$

27 → 9 → 3 → 1

$\log_b n = \dfrac{\log_a n}{\log_a b}$ $\log_2 n = \dfrac{\log_{10} n}{\log_{10} 2}$

$O(\log_2 n) = O\left(\dfrac{\log_{10} n}{\log_{10} 2}\right) = O(\log_{10} n)$

$$\underbrace{O(1)}_{\text{constant}} < \underbrace{O(\log n)}_{\text{log}} < \underbrace{O(n^1)}_{\text{linear}} < \underbrace{O(n \log n)}_{\text{linearthmic}} < \underbrace{O(n^2) < O(n^3) \ldots}_{\text{polynomial}} < \underbrace{1.1^n < 2^n \ldots}_{\text{exponential}}$$

$n - 10^9$ (billion)

$\log_{10} n = 9$

$n \begin{cases} O(n^2) \checkmark \\ O(n^3) \checkmark \end{cases}$

$n - O(\log n)$ ✗

$$\boxed{an^4 + bn^3 + cn^2 + dn + e - \text{polynomial}}$$

n^2 – polynomial of deg 2 – quadratic

n^3 – polynomial of deg 3 – cubic

$n^4, n^5 \ldots$ polynomial

$$O(n^{100000}) < O(1.1^n)$$

$n^{1000} = O(1.1^n)$ ✓✓

$1.1^n =$